ripple

XRPL WORKSHOP

Building on the XRP Ledger



Maxime Dienger
Senior Developer Advocate, Ripple
X: @krkmu_



Pamphile Tupui Roy
XRPL Architect, The Aha Company

THE XRP LEDGER

The XRP Ledger at a glance



Fast

Transactions settle with finality in 3–5 seconds.



Low cost

Fractions of a cent per transaction.



Proven

14+ years and 100M+ closed ledgers without interruption.



Decentralized

Validated by independent operators worldwide, with no central gatekeeper.



Sustainable

No mining, a very low, carbon-neutral energy footprint.



Open source

Open server code and client libraries in JS, Python, Java and more.

THE XRP LEDGER

What makes XRPL different



No mining

Federated consensus among trusted validators, no proof-of-work, no staking.



Native primitives

Payments, tokens, DEX and escrow are built into the protocol, no smart contract needed.



Account reserves

1 XRP base reserve plus 0.2 XRP per owned object.



Unified DEX and AMM

One shared order book and liquidity pools, open to every asset on the ledger.



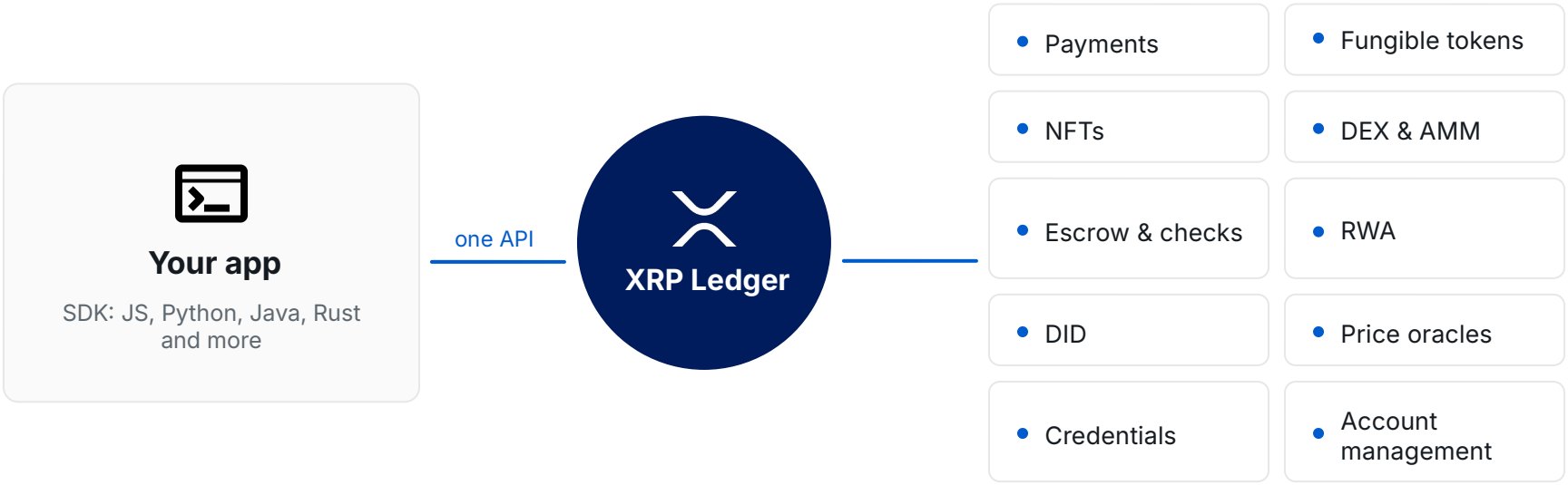
One API

Every feature sits behind the same JSON-RPC and WebSocket interface.

THE XRP LEDGER

One endpoint to do all the things

A few lines of JavaScript, Python or Java reach every feature of the ledger.



THE XRP LEDGER

Featured amendments

LIVE ON MAINNET

MPTokens XLS-33
A new fungible token standard.

TokenEscrow XLS-85
Escrow IOUs and MPTs, not just XRP.

Credentials XLS-70
On-chain identity attestations.

Permissioned Domains XLS-80
Credential-gated regulated spaces.

Batch XLS-56
Atomic bundles

OPEN FOR VOTING

Lending protocol + SAV XLS-65/66
Fixed-term loans from pooled vaults.

Permission delegation XLS-75
Delegate permissions to other accounts.

Sponsored fees & reserves XLS-68
A sponsor covers fees and reserves.

Confidential transfer XLS-96
Shielded MPT amounts on-ledger.

IN PROGRESS

Smart Escrow XLS-100
Programmable WASM escrow conditions.

Live status: livenet.xrpl.org/amendments

THE XRP LEDGER

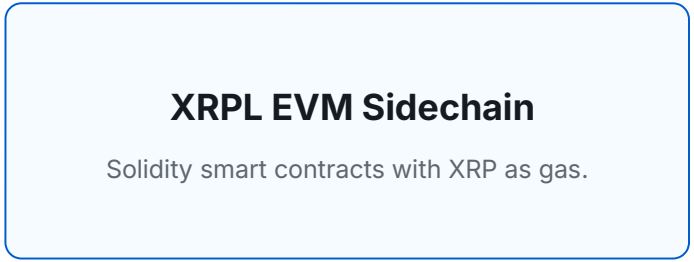
The extended ecosystem



Axelar

↔

bridges XRP and tokens



OTHER NETWORKS

Xahau
Hooks: small on-ledger logic attached to accounts.

The Root Network
Gaming and metaverse infrastructure using XRP for gas.

Evernode
Decentralized hosting layer built on Xahau.

Choose your network

PROD

XRPL Mainnet

The live network. Real XRP, real value, real users.

USE IT WHEN

Your app ships to real users and moves real value.

STAGING

XRPL Testnet

Replicates mainnet: the same active amendments, on a test environment.

USE IT WHEN

You build and test against exactly what mainnet runs today.

DEV

XRPL Devnet

Runs mainnet's amendments plus the ones open for voting.

USE IT WHEN

You want to try upcoming features before they activate.

Learn more: xrpl.org/docs/concepts/networks-and-servers/parallel-networks

Code examples

Your first transaction in four steps

CODE EXAMPLES

Client libraries

What is a client library?

A language-specific package that provides prebuilt methods and typed models to simplify interacting with an API.

xrpl.js
JavaScript / TypeScript
Recommended

xrpl-py
Python
Recommended

xrpl4j
Java

xrpl-rust
Rust

Community libraries
Go, PHP, Ruby and more

CODE EXAMPLES

Four steps to your first transaction

01

Connect to a node

Open a WebSocket to mainnet, testnet or devnet.

02

Create or import a wallet

Generate a keypair; on testnet, fund it from the faucet.

03

Prepare and send

Build the transaction JSON, sign it and submit.

04

Read the result

Check the engine result and transaction hash.

CODE EXAMPLES · 1/4

Step 1: connect to a node

```
import { Client } from "xrpl" // Public testnet node, free, no API key const client = new Client("wss://s.altnet.rippletest.net:51233") await client.connect()
```

Public networks

Mainnet
wss://xrplcluster.com

Testnet
wss://s.altnet.rippletest.net:51233

Devnet
wss://s.devnet.rippletest.net:51233

Step 2: create a wallet

```
// Generate and fund a wallet via the testnet faucet
const { wallet } = await  client.fundWallet()

console.log(wallet.address)   // "rf1BiG..." console.log(wallet.seed)      //
"sEd7ro..."

// Or import an existing account
const wallet = Wallet.fromSeed("sEd7...")
```

Good to know

The faucet funds each testnet wallet with test XRP.

An account is only activated on ledger once it holds the 1 XRP base reserve.

The seed is the secret, never commit it, never share it.

Step 3: prepare and send

```
const tx = {
 TransactionType: "Payment",
 Account: wallet.address,
 Destination: "rPT1Sjq2YGrBMTttX4GZHjKu9dyfzbpAYe",
 Amount: xrpToDrops("10"),  // 10 XRP, in drops
}
const result = await client.submitAndWait(tx, {
 autofill: true,
 wallet,
})
```

submitAndWait

Autofills Fee, Sequence and LastLedgerSequence, signs locally.

Resolves once the tx is in a validated ledger, usually 3–5 seconds.

Drops

1 XRP = 1,000,000 drops.

Sequence : your account's tx counter, prevents replay.

LastLedgerSequence : last ledger the tx can enter, then it expires.

Step 4: read the result

```
console.log(result.result.meta.TransactionResult)
// "tesSUCCESS"
console.log(result.result.hash)
// Look it up on any explorer
await client.disconnect()
```

Result codes

tes: success, applied to the ledger. **tec**: failed, but the fee was consumed.

Explorer: testnet.xrpl.org

Reference: xrpl.org/docs/references/protocol/transactions/transaction-results/tec-codes

CODE EXAMPLES

The entire code

```
import { Client, Wallet, xrpToDrops } from "xrpl"

const client = new Client("wss://s.altnet.rippletest.net:51233")   // 1. connect
await client.connect()

const { wallet } = await client.fundWallet()                       // 2. wallet

const result = await client.submitAndWait({                        // 3. send
 TransactionType: "Payment",
 Account: wallet.address,
 Destination: "rPT1Sjq2YGrBMTttX4GZHjKu9dyfzbpAYe",
 Amount: xrpToDrops("10"),
}, { autofill: true, wallet })

console.log(result.result.meta.TransactionResult)                  // 4. result
await client.disconnect()
```

Tokens on XRPL

Two standards: IOUs and multi-purpose tokens

TOKENS ON XRPL

Two token standards



SINCE 2012

IOU

A promise between parties. Tokens live on bilateral trustlines: each holder declares how much they trust each issuer, per currency.

The historical standard that powers the DEX, the AMM and years of issued currencies.

New standard



XLS-33 · LIVE ON MAINNET

Multi-purpose token

A single issuance object holders simply opt into. On-chain metadata, an enforced supply cap and richer issuer controls, without trustline complexity.

Designed for stablecoins, RWAs and everyday fungible tokens.

TOKENS ON XRPL

IOUs and trustlines

01 **An issuer creates a token**

An entity backs the token's value and is responsible for redemption, USD, EUR, anything.

02 **Holders set a trustline**

A trustline says "I trust this issuer up to this amount", one per issuer and currency.

03 **Tokens move as promises**

Holding a token means holding the issuer's promise, the concept behind stablecoins like RLUSD or USDC.



Real-world example

When you hold RLUSD on XRPL, you hold Ripple's promise to redeem 1 USD per token.

You can only receive it after establishing trust with Ripple as the issuer.

TOKENS ON XRPL

Multi-purpose tokens

01 **The issuer defines the token once**

One issuance object holds the supply cap, transfer rules and metadata: ticker, name, icon.

02 **Holders opt in by ID**

A single authorize transaction: no trust limits to size, no per-currency bookkeeping.

03 **Balances stay simple**

No rippling through intermediate accounts, so balances behave the way builders expect.

Built for issuers and holders

Issuers get enforced maximum supply, on-chain metadata, transfer fees, lock and clawback flags, and require-auth for regulated assets.

Holders get a simpler experience: one opt-in, predictable balances, no trustline settings to manage.

What is rippling?

With IOUs, a payment can flow through intermediate trustlines and shift balances on accounts along the way. Powerful for pathfinding, surprising if flags are misconfigured. MPTs never ripple.

TOKENS ON XRPL

IOU vs MPT

	IOU	MPT
Model	Bilateral trustlines, one per issuer and currency	Single issuance object, holders opt in by ID
Metadata	3-character currency code only	On-chain metadata: ticker, name, icon, docs
Supply	No native cap	Maximum supply enforced by the ledger
Rippling	Yes, needs careful flag configuration	None, simple, predictable balances
Issuer controls	Freeze, authorized trustlines	Lock, require-auth, transfer fees, clawback
DEX & AMM	Fully supported, deep integrations	On the roadmap; TokenEscrow works today



Building something new? Start with MPTs, the standard XRPL tooling is converging on. Reach for IOUs when you need DEX and AMM support today.

Issuing a token: IOU vs MPT

IOU

```
// The token lives on trustlines: balances are
// tracked on RippleState entries, per holder.
// One-time issuer account setup
{ TransactionType: "AccountSet",
 SetFlag: 8 } // asfDefaultRipple

// Issue = send a Payment to a holder
Amount: {
 currency: "USD",
 issuer: issuer.address,
 value: "100",
}
```

MPT

```
// The token lives in its own ledger object:
// one MPTokenIssuance, shared by all holders.
await client.submitAndWait({
 TransactionType: "MPTokenIssuanceCreate",
 Account: issuer.address,
 AssetScale: 2,
 MaximumAmount: "1000000",
 MPTokenMetadata: "...", // ticker, name, icon
}, { autofill: true, wallet: issuer })

// Issue = send a Payment to a holder
Amount: { mpt_issuance_id: id, value: "100" }
```

Accepting a token: IOU vs MPT

IOU

```
// Opt in with a trustline, one per issuer and currency
await client.submitAndWait({
 TransactionType: "TrustSet",
 Account: holder.address,
 LimitAmount: {
  currency: "USD",
  issuer: issuer.address,
  value: "1000000", // trust limit
 },
}, { autofill: true, wallet: holder })
```

MPT

```
// Opt in with the issuance ID, one transaction
await client.submitAndWait({
 TransactionType: "MPTokenAuthorize",
 Account: holder.address,
 MPTokenIssuanceID: id,
}, { autofill: true, wallet: holder })
```

Get RLUSD on testnet

01 **Open tryrlusd.com**

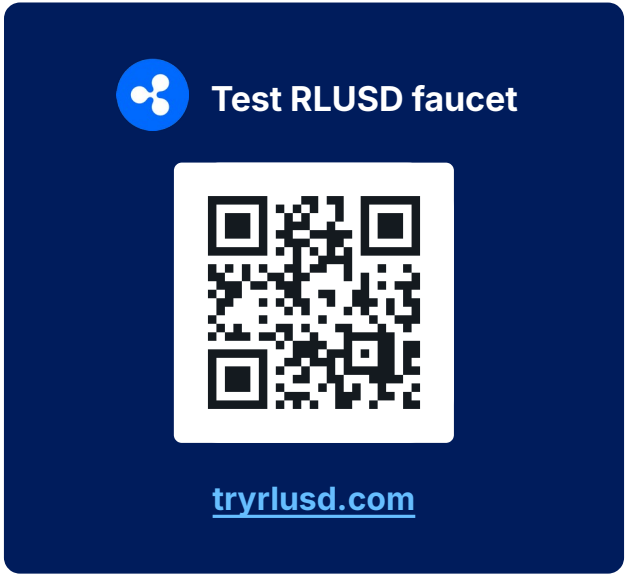
Scan the QR code or type the address, no sign-up needed.

02 **Connect your wallet**

The trustline to the RLUSD issuer is set up for you.

03 **Receive test RLUSD**

Now you can send, receive and exchange RLUSD on XRPL Testnet.



TOOLING

Choose your wallet



Xaman

Mobile app

xaman.app



Crossmark

Browser extension

crossmark.io



Privy

Embedded Wallet

privy.io



Joey

Mobile app

joeywallet.xyz



Wallet-as-a-Service (Palisade)
Embed secure XRPL wallets in your product with an API · docs.ripple.com/products/wallet



Ripple Custody
Institutional digital asset custody technology · ripple.com/solutions/digital-asset-custody

and more!

TOOLING

Builder toolkit

xrpl-connect

One SDK to rule all wallets: Xaman, Crossmark, GemWallet and more behind a single interface.

github.com/XRPL-Commons/xrpl-connect

Amendment testing suite

Try new amendments, MPT, escrow and more, against live networks from your browser.

tests.xrpl-commons.org

Scaffold-XRP

A full-stack starter kit: bootstrap an XRPL dApp with wallet connection and UI in minutes.

github.com/XRPL-Commons/scaffold-xrp

Ghostsig

A self-custodian PassKey wallet

ghostsig.dev

And more on the official docs: xrpl.org/resources/dev-tools

TOOLING

Reference app: Lending Protocol & SAV

Test the lending protocol and Single Asset Vault end to end, then fork the codebase to build your own.

LIVE DEMO

Try the demo

Create vaults, deposit, originate and repay loans, and inspect every state transition on-chain.

lending.xls-demo.com



OPEN SOURCE

Fork the codebase

The full reference implementation, ready to fork as the starting point for your own app.

github.com/ripple/xrpl-reference-app-lending-sav



TOOLING

AI agents on the XRPL

XRPL AI Hub

List your x402 service at xrpl-ai.org/join/service

xrpl-ai.org

t54

Agentic payment infrastructure: tooling for AI agents to transact safely.

t54.ai

XRPL Docs MCP server

Plug the XRPL documentation into Claude, Cursor and other AI tools.

mcp.xrpledger.ai

XRPL AI starter kit

Agentic transactions: get an AI agent sending its first XRPL transaction.

xrpl.org/docs/agents/agentic-transactions

AI AGENTS

x402: pay-per-request APIs with t54

1. **Agent calls an endpoint**
   A plain HTTP request to a paid API.
2. **Server answers 402**
   Price, pay-to address and network in the response.
3. **Agent signs an XRPL Payment**
   In XRP or RLUSD, sent with the retried request.
4. **Facilitator settles, server delivers**
   The t54 facilitator verifies and submits on-ledger.



Quickstart on xrpl.org
xrpl.org/docs/agents/agentic-payments-x402

SDKs to use

Python — x402-xrpl
Reference implementation by t54: FastAPI middleware for merchants, requests client for agents.

TypeScript — x402-xrpl & x402-xrpl/express
Merchant middleware for Express and a TypeScript buyer client.

Hosted facilitators — t54
xrpl-facilitator-testnet.t54.ai xrpl-facilitator-mainnet.t54.ai

Docs: xrpl-x402.t54.ai/docs

AI AGENTS

XRPL AI Hub: services and payments in one place

The public index and directory where agents discover services, pay in XRP or RLUSD, and verify settlement.

1. **Live service directory**
   Browse live services, merchants and settlement activity.

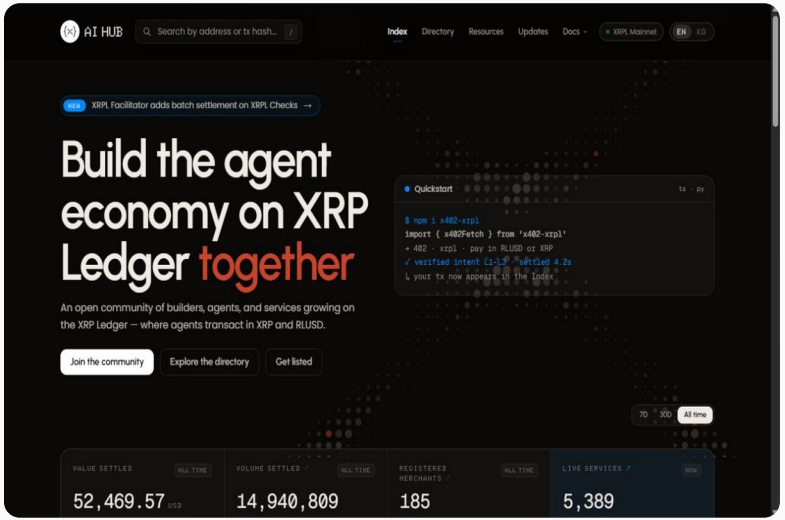
2. **Verifiable settlement**
   Every payment in the Index is an XRPL transaction you can look up by hash on any explorer.

3. **Self-service listing**
   List a live x402 endpoint at xrpl-ai.org/join/service; it appears in the Index automatically.

List a live x402 service

xrpl-ai.org/join/service

LIVE INDEX



14.9M payments 185 merchants 5,000+ services settles in 3–5 s

AI AGENTS

t54: spending controls that live outside the agent

Owners define the spending policy. t54 checks it before settlement.

Claw Credit

CREDIT FOR AGENT PAYMENTS

claw.credit

npm i @t54-labs/clawcredit-sdk

Agents can pay x402 services, including XRPL services priced in RLUSD. The owner signs a mandate that sets the payment cap, daily total, permitted merchants and assets, and expiry. Every payment must pass those limits before settlement.

Beta. Registration requires an invite code. Codes are available for teams.

x402 Secure

FACILITATOR RISK CHECK

t54.ai/x402-secure

Available on testnet and mainnet. Payments carrying Verifiable Intent receive an allow or deny decision from Trustline before settlement. Payments without Verifiable Intent continue through the standard x402 flow.

Trustline

RISK AND UNDERWRITING

portal.t54.ai

Trustline powers the risk and underwriting decisions behind both products. The self-service sandbox lets builders create an organization, get an API key and request a real-time assessment with audit evidence. Production access requires KYB.

HOW IT FITS THE MVP

AUTONOMOUS XRPL TRANSACTION

x402 through the hosted XRPL facilitator

LIMITS OUTSIDE THE AGENT

Owner-signed mandate checked before settlement

HUMAN CONTROL OVER LIMITS

The owner signs a new mandate. Each mandate expires.

Docs for all three: t54.ai/docs

AI AGENTS

MPP: machine payments on XRPL

mpp.dev/payment-methods/xrpl

`npm i xrpl-mpp-sdk` extends mppx, built on xrpl.js · Beta

Charge

ONE PAYMENT PER REQUEST

How	The agent signs a Payment, the server submits and verifies it. Final in 3 to 5 s.
Assets	XRP, IOUs and MPTs
Best for	Single API calls, content access, one-off purchases

```
xrpl.charge({
  recipient, network:
'testnet' ,
  currency:
'XRP', // or IOU / MPT })
```

Session

PAYMENT CHANNEL

How	Deposit once on-chain, then off-chain vouchers. One closing transaction settles them all.
Assets	XRP only
Best for	LLM APIs, metered services, per-token billing

```
xrpl.channel({
  recipient, network:
'testnet' ,
  autoClose: { idleMs:
30_000  },
})
```

AI AGENTS

x402 or MPP?

	x402	MPP
Driven by	x402 Foundation, open protocol	Tempo and Stripe
XRPL SDK	x402-xrpl (t54)	xrpl-mpp-sdk (beta)
Settlement	A facilitator verifies and submits	Your server verifies and submits, no facilitator
Payment model	One on-chain payment per request	Charge per request, or session over a Payment Channel
Assets	XRP, RLUSD and other IOUs	Charge: XRP, IOUs, MPTs. Session: XRP
Pick it for	Joining the existing x402 ecosystem of paid APIs	High-frequency, metered billing such as LLM tokens

Wallets for your agent

Both work with either protocol, and keep the raw key away from the agent.

OWS

Open Wallet Standard

Local, policy-gated signing. One wallet, addresses on every chain including XRPL.

github.com/open-wallet-standard

Privy

Server-side agent wallets on XRPL, with a policy engine and built-in x402 and MPP clients.

docs.privy.io

ripple

X: @krkmu_

THANK YOU

Resources

DevRel

All the resources to get started
linktr.ee/rippledevrel

XRPL resource index
github.com/RippleDevRel/xrpl-ressource-index

XRPL documentation

Concepts, references, tutorials and dev tools.
xrpl.org/docs